

LEA LABS · DIGITAL SKILLS

Web Development Fundamentals

A practical course pack for building, testing, and explaining modern web experiences.

Designed for teaching and learning

Code samples · guided practice · testing habits · inclusive interfaces

LEARN · BUILD · PUBLISH

LEA LABS

Web Development Fundamentals

A university-style teaching and learning pack

Version: 1.0

Audience: Beginner-to-intermediate digital skills learners

Recommended delivery: 12–14 weeks, 3 hours of guided instruction plus 3–5 hours of independent practice each week

Project spine: Campus Connect, a responsive student events board built with HTML, CSS, and JavaScript

Author: Manus AI for LEA Labs

Teaching promise. Students will not only explain what HTML, CSS, and JavaScript are. They will design, build, test, document, and publish a small web experience that other people can use.

How to use this pack

This pack is designed as a bridge between classroom instruction and independent making. Each module begins with a concept, moves into a guided demonstration, and ends with evidence that the learner can apply the concept in a working page. The teacher should model an imperfect first attempt, narrate decisions, and ask learners to inspect the browser rather than treating code as magic. The student should keep a learning journal, make small commits, and explain the reason behind each change.

The pack uses **Campus Connect** as a continuous project. The project is intentionally small enough to finish but rich enough to exercise semantic HTML, responsive CSS, DOM manipulation, filtering, form validation, accessibility, debugging, Git, and deployment. The final application is not a production booking system: it is a browser-side learning project. Any real system would add server-side validation, authentication, secure storage, privacy controls, and a database.

Learning outcomes

By the end of the course, a successful learner can:

Outcome	Evidence of learning
Explain how a browser turns a URL into a rendered page	A labelled request-to-render diagram and a short oral explanation
Create semantic, valid HTML	A structured page using appropriate landmarks, headings, links, forms, and articles
Use CSS to create a responsive interface	A layout that works at mobile, tablet, and desktop widths without horizontal scrolling
Use JavaScript to model data and respond to events	A rendered event list, category filters, and a form interaction
Improve accessibility and usability	Keyboard-accessible controls, visible focus, labels, status messages, and readable contrast
Debug systematically	A written reproduction-hypothesis-test-fix record for at least three defects
Use Git and publish a small web project	A repository with meaningful commits, a README, and a working deployment
Communicate technical decisions	A short project walkthrough explaining trade-offs and next steps

Delivery rhythm

A strong session follows a repeatable rhythm. Start with a concrete question, demonstrate one concept in a small example, let learners predict what will happen, then change the code and inspect the result. Finish by asking learners to make one meaningful variation. A useful weekly pattern is shown below.

Segment	Typical time	Teacher action	Student action
Orient	15 min	Connect the lesson to the project goal	State the problem in their own words
Explain	30 min	Introduce one concept and its vocabulary	Annotate examples and ask questions
Build together	45 min	Code slowly and narrate decisions	Type, run, inspect, and compare
Practise	45 min	Circulate and coach with questions	Modify the project independently
Review	20 min	Use a checklist and peer feedback	Demonstrate evidence and record next steps

Module 1 — The web as a system

1.1 The browser is a client

A web page is the result of a conversation between a client and a server. The browser is the client: it sends a request for a resource, receives a response, interprets the response, and presents a usable interface. The server may return HTML, CSS, JavaScript, images, JSON, or an error response. The browser then performs work locally, including parsing HTML into a document tree, applying CSS rules, executing JavaScript, and painting pixels.

The separation between the network and the browser matters. A page can look correct while a network request fails, or a request can succeed while the browser refuses to execute a script because of a syntax or policy error. Learners should therefore inspect both the rendered page and the browser's developer tools.

```
flowchart LR
    U[User enters a URL] --> B[Browser creates request]
    B --> S[Server receives request]
    S --> R[Server returns response]
    R --> H[Browser parses HTML]
    H --> C[Browser applies CSS]
    C --> J[Browser runs JavaScript]
    J --> P[Browser paints page]
    P --> U2[User interacts]
    U2 --> B
```

Teacher guidance. Ask learners to open the Network panel and reload a page. Have them identify the document request, stylesheet requests, script requests, status codes, and approximate timing. Do not begin with protocol detail. Begin with the visible question: “Which pieces had to arrive before this page could be used?”

Student checkpoint. Write a five-sentence explanation of what happens after a user enters a URL. Include the words **request**, **response**, **HTML**, **CSS**, and **JavaScript**.

1.2 URLs, paths, and status codes

A URL identifies a resource. The scheme describes how the resource is accessed, the host identifies the destination, and the path identifies a resource on that host. Query parameters can provide additional input. HTTP status codes give the browser and developer a high-level result: **2xx** indicates success, **3xx** indicates redirection, **4xx** indicates a client-side request problem, and **5xx** indicates a server-side problem. These categories are useful diagnostic signals, not complete explanations.

Code family	Meaning	Student interpretation
200-299	Successful response	The server returned the requested result
300-399	Redirection or caching-related response	Follow the next location or inspect cache behavior
400-499	Problem with the request or permissions	Check the URL, authentication, or request data
500-599	Server-side failure	Check server logs, dependency health, or upstream services

Never teach learners that a **200** response proves the interface is correct. The HTML could be malformed, the CSS could be missing, and JavaScript could fail after the response arrives.

Module 2 — HTML gives content meaning

2.1 The document frame

HTML is a language for describing the structure and meaning of content. A useful document begins with a doctype, a language declaration, a metadata section, and a body. The viewport declaration tells mobile browsers how to use the available width; without it, a page may be laid out as if it were a wide desktop page and then scaled down.

```

<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Campus Connect</title>
    <link rel="stylesheet" href="styles.css">
    <script src="script.js" defer></script>
  </head>
  <body>
    <header>
      <a href="/" aria-label="Campus Connect home">Campus Connect</a>
      <nav aria-label="Primary navigation">
        <a href="#events">Events</a>
        <a href="#reserve">Reserve</a>
      </nav>
    </header>

    <main>
      <section aria-labelledby="page-title">
        <p>Find your next useful connection.</p>
        <h1 id="page-title">Events that move campus life forward.</h1>
      </section>

      <section id="events" aria-labelledby="events-title">
        <h2 id="events-title">Upcoming events</h2>
        <div id="filters" aria-label="Filter events"></div>
        <div id="events-grid"></div>
      </section>

      <section id="reserve" aria-labelledby="reserve-title">
        <h2 id="reserve-title">Reserve a place</h2>
        <form id="reserve-form">
          <label for="name">Name</label>
          <input id="name" name="name" autocomplete="name" required>

          <label for="email">Email</label>
          <input id="email" name="email" type="email" autocomplete="email" required>

          <button type="submit">Reserve</button>
          <p id="status-message" role="status" aria-live="polite"></p>
        </form>
      </section>
    </main>

    <footer><small>Built for the Campus Connect learning project.</small></footer>
  </body>
</html>

```

Semantic landmarks

Semantic elements communicate purpose. A `header` introduces a page or section; `nav` groups navigation; `main` contains the unique primary content; `section` groups a topic with a heading; `article` represents a self-contained item; `form` groups controls that collect user input; and `footer` contains closing information. These elements help users, assistive technologies, search engines, and future developers understand the page.

A `div` is not wrong. It is useful when no more specific element fits. It becomes a problem when it replaces a meaningful control or landmark. A clickable `div` does not automatically behave like a button for keyboard users, does not expose the same semantics, and often requires extra event and focus logic.

Headings and reading order

Headings are a structural outline, not merely large text. A page normally begins with one `h1`, then uses `h2` headings for major sections and `h3` headings for subtopics inside those sections. Do not choose a heading level because its default font size looks attractive. Choose it because it expresses the content hierarchy; use CSS for visual scale.

Activity. Ask students to remove all CSS from their page. If the page still reads in a sensible order, the underlying HTML structure is probably doing useful work.

2.2 Links, buttons, and forms

A link moves the user to another resource or location. A button performs an action on the current page. A form collects information. The distinction is central to predictable interaction. A link styled like a button is still a link; this is acceptable when it navigates. A button styled like a link is still a button; this is acceptable when it changes application state.

Labels must be associated with form controls. The most straightforward pattern is a `label` with a `for` attribute matching the control's `id`. Placeholder text is not a replacement for a label because it disappears when the user begins typing and may be difficult to perceive.

Native HTML validation provides useful first feedback. Attributes such as `required`, `type="email"`, `minlength`, `maxlength`, `min`, and `max` allow the browser to detect some invalid input before submission. However, client-side validation can be bypassed and is not a security boundary. A real server must validate and authorize received data independently [1].

Module 3 — CSS turns structure into experience

3.1 The cascade and the box model

CSS rules consist of selectors and declarations. The cascade determines which declarations apply when several rules target the same element. A browser considers origin, importance, specificity, and source order. Beginners often solve a styling conflict by adding a more specific selector, but excessive specificity makes future changes difficult. Prefer simple class-based selectors and a predictable stylesheet order.

Every element participates in a box model. Its content area can have padding, a border, and a margin. The following reset makes declared widths easier to reason about because padding and borders are included within the element's declared size.

```
*,
*::before,
*::after {
  box-sizing: border-box;
}

:root {
  --bg: #ffffdf;
  --ink: #151116;
  --purple: #4d176e;
  --deep-purple: #1f0d2e;
  --orange: #f47945;
  --lavender: #f6eef9;
  --cream-orange: #fff3ed;
  --space-1: 0.5rem;
  --space-2: 1rem;
  --space-3: 1.5rem;
  --space-4: 2.5rem;
  --measure: 70rem;
}

body {
  margin: 0;
  background: var(--bg);
  color: var(--ink);
  font-family: Outfit, system-ui, sans-serif;
  line-height: 1.55;
}

.container {
  width: min(100% - 2rem, var(--measure));
  margin-inline: auto;
}
```

Teacher demonstration. Toggle `box-sizing`, `padding`, `border`, and `margin` in DevTools. Ask students to predict the final rendered width before each change. The goal is not memorizing a diagram; it is learning to inspect the source of unexpected space.

3.2 Grid and Flexbox

Flexbox is primarily a one-dimensional layout method. It is useful for a row of navigation links, a button group, or aligning a label and an icon. Grid is a two-dimensional layout method. It is useful for the event-card area because the layout has both rows and columns. Both can be responsive, but neither removes the need to think about content, line length, and narrow widths [2].

```
.site-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: var(--space-3);
}

.site-nav {
  display: flex;
  flex-wrap: wrap;
  gap: var(--space-2);
}

.event-grid {
  display: grid;
  gap: var(--space-3);
  grid-template-columns: repeat(auto-fit, minmax(17rem, 1fr));
}
```

The `auto-fit` pattern allows the browser to decide how many columns can fit while respecting a minimum card width. This is often more resilient than writing a media query for every device. Add a media query when the content itself needs a different arrangement.

```
.reservation-form {
  display: grid;
  gap: var(--space-2);
  grid-template-columns: 1fr 1fr auto;
  align-items: end;
}

@media (max-width: 42rem) {
  .reservation-form {
    grid-template-columns: 1fr;
  }
}
```

3.3 States, focus, and readable contrast

Interactive controls have states: default, hover, focus, active, disabled, valid, and invalid. The focus state is essential for keyboard users. Never remove the browser's focus outline without replacing it with a visible equivalent.

```
button,
.filter-btn {
  border: 2px solid var(--purple);
  background: var(--purple);
  color: white;
  padding: 0.7rem 1rem;
  font: inherit;
  cursor: pointer;
}

button:hover,
.filter-btn:hover {
  background: var(--orange);
  border-color: var(--orange);
}

button:focus-visible,
.filter-btn:focus-visible,
input:focus-visible {
  outline: 3px solid var(--orange);
  outline-offset: 3px;
}

input:invalid:user-invalid {
  border-color: #a52f2f;
}
```

The exact colour choice must be checked against the background and text size. A brand palette is not automatically an accessible palette. If a colour does not provide enough contrast for body text, use it for a border, large text, or decorative emphasis instead.

Module 4 — JavaScript gives the page behaviour

4.1 Values, arrays, objects, and functions

JavaScript lets a page make decisions and respond to events. In Campus Connect, an array of objects models the events. Each object holds related values, which makes the data easy to filter and render.

```
const events = [
  {
    id: 1,
    title: "Build with the Web",
    category: "workshop",
    date: "2026-09-03",
    description: "A guided practice session for HTML, CSS, and JavaScript."
  },
  {
    id: 2,
    title: "Portfolio Review Circle",
    category: "career",
    date: "2026-09-10",
    description: "Bring one project and receive structured peer feedback."
  }
];

function countByCategory(items, category) {
  return items.filter((item) => item.category === category).length;
}
```

Functions name a repeatable operation. A function should have one clear responsibility whenever possible. If a function both reads form data, filters events, updates several unrelated sections, and saves to storage, it is difficult to test and change. Split responsibilities as the project grows.

4.2 The DOM and safe rendering

The Document Object Model is the browser's object representation of the HTML document. JavaScript can find nodes, read values, create elements, update text, and attach event listeners. When content comes from a user or an external service, prefer `textContent` and explicit element creation rather than inserting unsanitized HTML.

```
const grid = document.querySelector("#events-grid");

function renderEvents(items) {
  grid.replaceChildren();

  items.forEach((event) => {
    const article = document.createElement("article");
    article.className = "event-card";

    const heading = document.createElement("h3");
    heading.textContent = event.title;

    const meta = document.createElement("p");
    meta.textContent = `${event.category} · ${event.date}`;

    const description = document.createElement("p");
    description.textContent = event.description;

    article.append(heading, meta, description);
    grid.append(article);
  });
}

renderEvents(events);
```

This approach is more verbose than one template string, but it makes the relationship between data and DOM nodes explicit and avoids treating data as trusted markup. If a project uses template strings, the developer should still sanitize or escape any untrusted content.

4.3 Events and filtering

An event is a signal that something happened, such as a click, key press, form submission, or page load. `addEventListener()` registers a function to run when a particular event occurs [3]. The event object can provide the target element and other context.

```
const filterButtons = document.querySelectorAll("[data-category]");

filterButtons.forEach((button) => {
  button.addEventListener("click", () => {
    const category = button.dataset.category;
    const visibleEvents = category === "all"
      ? events
      : events.filter((event) => event.category === category);

    filterButtons.forEach((item) => {
      item.setAttribute("aria-pressed", String(item === button));
    });

    renderEvents(visibleEvents);
  });
});
```

The `data-category` attribute lets the HTML declare which category a button represents. The JavaScript reads that value rather than relying on button order. The `aria-pressed` state tells assistive technologies whether a toggle-style control is selected. If the filter is a single-choice tab pattern, the interaction model and ARIA roles should be designed consistently; do not add ARIA attributes without understanding the interaction they describe.

4.4 Form submission and feedback

A form's `submit` event is the correct place to handle a demo reservation. The browser performs built-in validation before firing a successful submit event. The handler can prevent navigation, read the form through `FormData`, and write a clear status message.

```
const form = document.querySelector("#reserve-form");
const statusMessage = document.querySelector("#status-message");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  const data = new FormData(form);
  const name = String(data.get("name") || "").trim();

  statusMessage.textContent = `Thanks, ${name}. Your demo reservation is recorded locally.`;
  form.reset();
});
```

This demo does not send or save a real reservation. That limitation should be visible in the interface and documentation. A production reservation flow would require a trusted server, server-side validation, authentication or abuse controls where appropriate, safe persistence, privacy information, and a reliable confirmation mechanism.

Module 5 — Accessibility is part of correctness

Accessibility is not a decorative layer added after the page is finished. It is part of whether the interface works. Semantic HTML provides a foundation, but the developer must also check keyboard order, focus visibility, labels, headings, status messages, colour contrast, text zoom, and motion preferences.

Practical accessibility audit

Check	What to do	Evidence
Keyboard	Use Tab, Shift+Tab, Enter, and Space without a mouse	Every interactive element is reachable and understandable
Focus	Observe the active element while tabbing	Focus is visible and not hidden behind content
Names	Inspect links, buttons, and inputs	Visible labels or accessible names explain purpose
Structure	Inspect headings and landmarks	The page still makes sense without styling
Contrast	Use a contrast checker and visual judgment	Body text and controls remain readable
Zoom	Test at 200% browser zoom	Content does not disappear or require two-dimensional scrolling
Status	Submit and filter using keyboard	Feedback is announced or visibly placed near the action

Teacher activity. Pair learners. One learner uses the page with the mouse; the other uses only the keyboard and records every moment of confusion. Swap roles. This makes accessibility an observed user experience rather than an abstract compliance list.

Module 6 — Debugging and quality

6.1 A repeatable debugging method

When something breaks, avoid random edits. First reproduce the problem consistently. Then describe the expected behavior and the actual behavior. Form one hypothesis, inspect the relevant evidence, make the smallest testable change, and verify that the change fixed the problem without breaking another behavior.

Step	Question	Example
Reproduce	Can I make the failure happen again?	Clicking “workshop” shows no cards
Observe	What exactly is wrong?	The URL has no error, but the grid is empty
Hypothesize	What is the most likely cause?	The data category is <code>workshops</code> , the button says <code>workshop</code>
Inspect	Which evidence can test the hypothesis?	Log <code>event.category</code> and <code>button.dataset.category</code>
Change	What is the smallest safe fix?	Normalize the category value
Verify	Does the fix work and preserve other behavior?	Test all filters and the reset state
Record	What will help future work?	Add a short note to the project log

6.2 Common failure modes

The stylesheet does not load. Inspect the Network panel and confirm the path and case match the file name. Check that the link is in the document head and that the server is serving the correct directory.

The script runs before the HTML exists. Use `defer`, place the script at the end of the body, or initialize after `DOMContentLoaded`. Do not hide timing problems by adding arbitrary delays.

The cards are empty. Check that the array exists, that the render function is called, that the selected DOM node is not `null`, and that the filter returns items.

The form reloads the page. Confirm that the submit handler is attached to the form, not only the button, and that `event.preventDefault()` is called in the demo flow after browser validation has passed.

The page overflows on mobile. Inspect long words, fixed widths, images, grids, and form rows. Use flexible units, `max-width: 100%` for media, and a breakpoint where the content actually needs to stack.

6.3 Console discipline

A clean console is not the same as a correct application, but errors and warnings are evidence. Students should not submit a project with unresolved errors that they have not documented. Teach them to read the first meaningful error, open the file and line number, and work backward from the failing assumption.

Module 7 — Version control and publishing

Git records deliberate changes to a project. A useful beginner workflow is:

```
git init
git add index.html styles.css script.js
git commit -m "Create semantic Campus Connect page"

# after a focused improvement
git add .
git commit -m "Add responsive event grid"
```

A commit should describe a coherent change. “Update stuff” is difficult to understand later; “Add category filters” gives the team a useful history. Students should commit after a stable milestone, not after every keystroke and not only at the end.

A README should state what the project does, how to run it locally, what was learned, what limitations remain, and how to test the major interactions. If the page is published, the README should link to the deployment and note whether the form is a demo.

Before publishing, remove secrets, test the production path, inspect the deployed site on a narrow viewport, and verify that all referenced files are present. A static page can be published through a static hosting service, but any real server-side behavior must be treated as a separate architectural decision.

Course project — Campus Connect

Functional specification

Campus Connect is a browser-based events board for learners. It must show a short introduction, a set of event cards, category filters, and a reservation form. The page must work without a framework so that the learner can see the relationship between the document, the stylesheet, and the script.

Minimum features

Feature	Required behavior
Event data	At least six events represented as JavaScript objects
Event cards	Each card shows title, category, date, and description
Filters	“All” plus at least three categories; filtering updates the visible cards without page reload
Reservation form	Name and valid email are required; successful submit shows a visible status message
Responsive layout	Works at 320px, tablet width, and desktop width
Accessibility	Semantic landmarks, labels, visible focus, keyboard operation, and readable status feedback
Documentation	README with setup, features, test checklist, limitations, and screenshots or a live link

Suggested implementation sequence

Milestone 1: Static structure. Create the HTML frame, headings, six static placeholders, filter buttons, and labelled form. Open the file in a browser and verify that the reading order makes sense without CSS.

Milestone 2: Visual system. Add CSS tokens, a container, typography, colour, the event-card grid, button states, and form layout. Test the page at a narrow width before adding polish.

Milestone 3: Data rendering. Replace repeated event markup with a JavaScript array and `renderEvents()` function. Use explicit DOM creation or a carefully controlled template approach.

Milestone 4: Filtering. Add data attributes to filter buttons. Register click events, compute a filtered array, re-render, and update the selected state.

Milestone 5: Reservation feedback. Add native validation, a submit event, `FormData`, and an `aria-live` status message. State clearly that the reservation is a local demo.

Milestone 6: Audit and publish. Run the keyboard, responsive, form, console, and README checks. Make a short demonstration video or live walkthrough for assessment.

Complete starter `script.js`

```
const events = [
  {
    id: 1,
    title: "Build with the Web",
    category: "workshop",
    date: "2026-09-03",
    description: "A guided practice session for HTML, CSS, and JavaScript."
  },
  {
    id: 2,
    title: "Portfolio Review Circle",
    category: "career",
    date: "2026-09-10",
    description: "Bring one project and receive structured peer feedback."
  },
  {
    id: 3,
    title: "Open Source Evening",
    category: "community",
    date: "2026-09-17",
    description: "Learn how to make a first contribution to a shared codebase."
  },
  {
    id: 4,
    title: "Design Systems Studio",
    category: "workshop",
    date: "2026-09-24",
    description: "Explore spacing, type scale, colour tokens, and reusable components."
  },
  {
    id: 5,
    title: "Meet the Makers",
    category: "community",
    date: "2026-10-01",
    description: "A conversation with people building practical digital products."
  },
  {
    id: 6,
    title: "CV and README Clinic",
    category: "career",
    date: "2026-10-08",
    description: "Turn your learning evidence into a clear professional story."
  }
];

const grid = document.querySelector("#events-grid");
const filterButtons = document.querySelectorAll("[data-category]");
const form = document.querySelector("#reserve-form");
const statusMessage = document.querySelector("#status-message");

function renderEvents(items) {
  grid.replaceChildren();

  if (items.length === 0) {
    const empty = document.createElement("p");
    empty.textContent = "No events match this category yet.";
    grid.append(empty);
    return;
  }

  items.forEach((event) => {
    const article = document.createElement("article");
    article.className = "event-card";

    const category = document.createElement("p");
    category.className = "event-category";
    category.textContent = event.category;
```

```
const heading = document.createElement("h3");
heading.textContent = event.title;

const date = document.createElement("p");
date.className = "event-date";
date.textContent = event.date;

const description = document.createElement("p");
description.textContent = event.description;

article.append(category, heading, date, description);
grid.append(article);
});
}

filterButtons.forEach((button) => {
  button.addEventListener("click", () => {
    const category = button.dataset.category;
    const visibleEvents = category === "all"
      ? events
      : events.filter((event) => event.category === category);

    filterButtons.forEach((item) => {
      item.setAttribute("aria-pressed", String(item === button));
    });

    renderEvents(visibleEvents);
  });
});

form.addEventListener("submit", (event) => {
  event.preventDefault();
  const data = new FormData(form);
  const name = String(data.get("name") || "").trim();
  statusMessage.textContent = `Thanks, ${name}. This demo recorded your interest locally.`;
  form.reset();
});

renderEvents(events);
```

Teacher extension ideas

A class that completes the minimum project can add a search field, event counts, a selected-event summary, a “favourites” interaction, a date sort, or `localStorage` persistence. More advanced learners can replace the local array with a JSON endpoint, but they must discuss loading states, errors, validation, and the security implications of trusting external data.

Teacher assessment guide

The assessment should reward working behavior, thoughtful structure, and the learner's ability to explain decisions. It should not reward visual complexity for its own sake. A simple page that is accessible, responsive, and well explained is stronger than a decorative page that fails on mobile.

Criterion	Weight	Exceeds expectations	Meets expectations	Developing
HTML structure and semantics	20%	Clear landmarks, heading hierarchy, meaningful controls, and strong reading order	Mostly semantic structure with minor inconsistencies	Heavy dependence on generic containers or missing labels
CSS layout and responsiveness	20%	Flexible layout, intentional tokens, strong states, no overflow at tested widths	Works across required widths with understandable CSS	Breaks at narrow widths or relies on fixed positioning
JavaScript behavior	25%	Clean data model, rendering, filtering, and form feedback with good separation	Required interactions work with minor code duplication	Interactions are incomplete, brittle, or unclear
Accessibility and UX	15%	Keyboard flow, visible focus, labels, contrast, and status feedback are deliberate	Most required checks pass	Mouse-only flow, missing labels, hidden focus, or unclear feedback
Debugging and testing	10%	Evidence-based test log and resolved console issues	Basic test checklist is complete	Testing is informal or defects remain unexplained
Documentation and communication	10%	README clearly explains purpose, setup, limitations, and decisions	README and demonstration cover the basics	Project is difficult to run or explain

Feedback prompts

The teacher can ask: “What user problem does this page solve?” “Which HTML element carries the most meaning here?” “What happens if the event array is empty?” “How do you know the filter is active for a keyboard user?” “Which parts are only a demo?” “What evidence supports your last fix?” These questions move assessment from code copying toward understanding.

Student reflection page

Write a short reflection after the project demonstration. Explain one decision you made about HTML structure, one decision you made about responsive CSS, and one decision you made about JavaScript behavior. Describe one bug you encountered, the hypothesis you tested, and the evidence that confirmed the fix. Finally, identify one limitation in the current project and one improvement you would make with more time.

References

- [1] [MDN Web Docs — Client-side form validation](#), explaining native validation, user feedback, and why server-side validation remains necessary.
- [2] [MDN Web Docs — Responsive web design](#), explaining fluid layouts, media queries, flexible grids, and mobile-first practice.

- [3] [MDN Web Docs — Introduction to events](#), explaining browser events, `addEventListener()`, event objects, and default behavior.
- [4] [MDN Web Docs — Learn web development](#), the broader learning pathway for HTML, CSS, JavaScript, accessibility, and tooling.